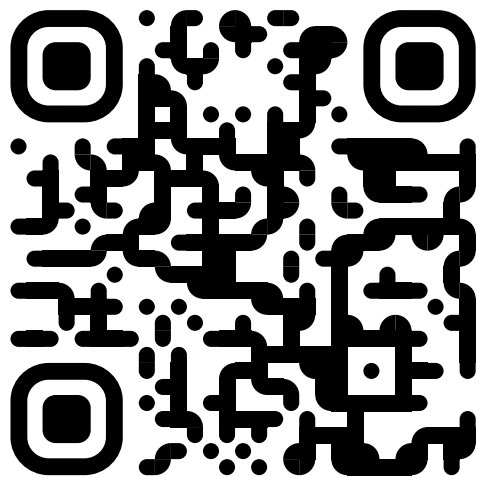




angrr

Auto Nix GC Roots Retention

Yinfeng

2025-12-27



, 本作品采用署名-非商业性使用-相同方式共享 4.0 协议国际版协议授权

关于我

Yinfeng



github.com/linyinfeng

其他 Nix 相关项目：

- **dotfiles** - 我的配置文件
- **commit-notifier** - NixOS CN 群的 PR 通知
- **nix-gc-s3** - 为 S3 cache 做 GC
- **flat-flake** - 扁平化 flake inputs 检查
- **oranc** - OCI registry as Nix cache (实验性)
- **conf2nix** - 从 Kconfig 生成 Nixpkgs 的 Linux structuredConfig (其实没啥用)

angrr == Auto Nix GC Roots Retention

一个自动清理 Nix GC roots 的服务/工具。

1. 管理 Nix profile 的 generation。

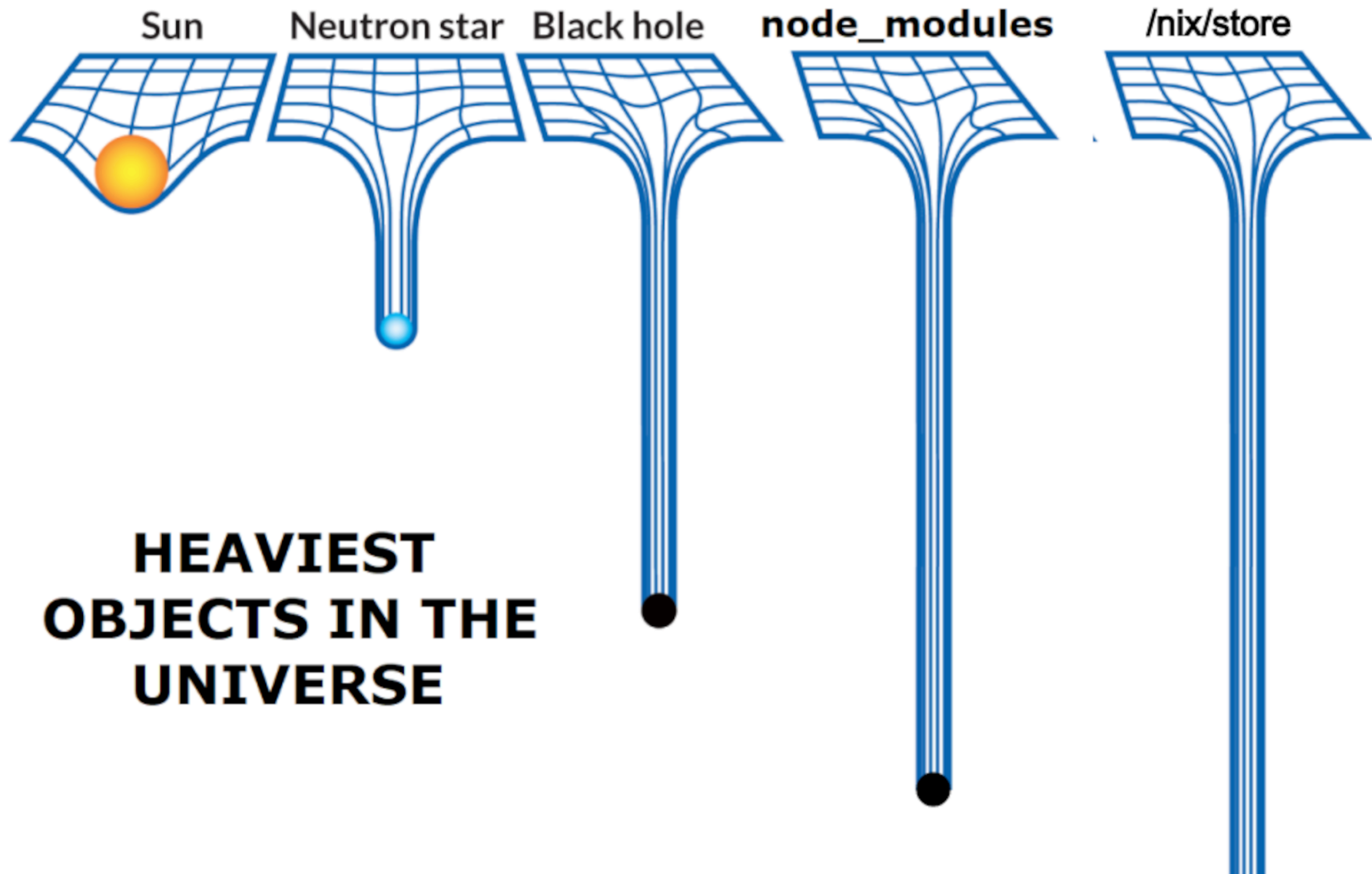
可以处理 `nix-env/nix profile`, `NixOS system`, `home-manager`, 等等。

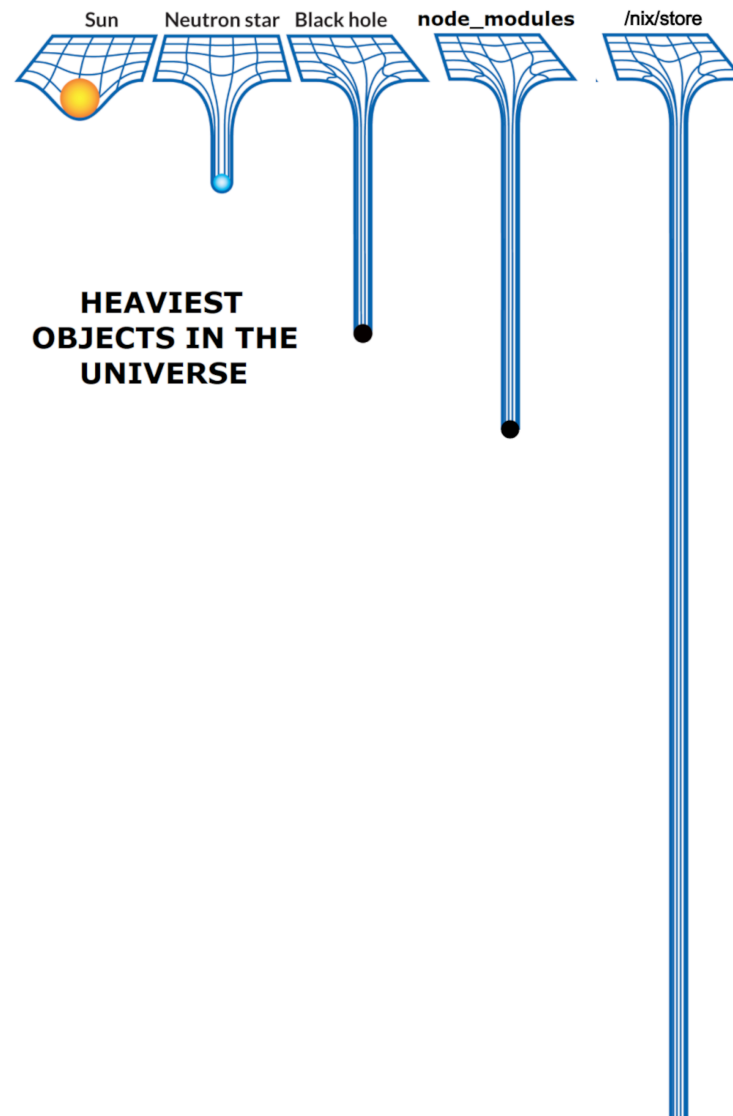
2. 管理项目使用的临时 GC roots。

包括 `nix-build/nix build`, `nix-direnv` 产生的 GC roots, 等等。

3.  所有策略**高度可定制**。

4.  作为服务**无感**运行。





为什么有 GC， /nix/store 还是这么大？

Nix 的 GC roots 有许多来源，且有些来源缺乏好的回收机制。

上页幻灯片中的图片来自于 [reddit - r/NixOS - /nix/store](#)，matthew-croughan 制作。

Nix 如何查找 GC root

```
1 nix-store --gc --print-roots
```

bash

Nix 做垃圾回收时使用的 GC roots 分两部分：

1. 运行时 GC roots，来自 procfs。
 - /proc/*/{exe,maps,environ,cwd,fd/*}
 - /proc/sys/kernel/{modprobe,fb splash,poweroff_cmd}
2. 磁盘上的 GC roots，位于 /nix/var/nix/{gcroots,profiles} 目录²下。
 - 递归查找该目录下的所有符号链接，指向的 store path³ 都是 GC roots

²其实还有神秘的 /nix/var/nix/temproots。

³名字与 store path 相同的空文件也是 GC root，hydra 会创建这样的文件。

程序如何创建 GC root

```
1 nix build nixpkgs#hello --out-link result
```

bash

1. 创建软链接指向 store path

```
./result -> /nix/store/2bcv91i...-hello-2.12.2
```

2. 在 /nix/var/nix/gcroots/auto 中创建软链接指向 out link

```
/nix/var/nix/gcroots/auto/l1zcgxj... -> $PWD/result
```


常见 GC Roots 来源

```
ls /nix/var/nix/gcroots/auto
```

```
bash
```

1. 系统 profile，如：

- /nix/var/nix/profiles/system-N-link

nixos-rebuild 等工具维护 /nix/var/nix/profiles/system profile，它的 generation 成为 GC roots。

可由 nixos-collect-garbage --delete-old/--delete-older-than 回收。

常见 GC Roots 来源

```
ls /nix/var/nix/gcroots/auto
```

 Bash

2. 用户 profile:

- ~/.local/state/nix/profiles/profile-N-link
- /nix/var/nix/profiles/per-user/root/profile-N-link

nix-env/nix profile 工具维护用户 profile。

如果是普通用户这个 profile 就在 `$XDG_STATE_HOME/nix/profile`，如果是 root 用户则在 `/nix/var/nix/profiles/per-user/root/profile`。

同样，可由 `nixos-collect-garbage --delete-old/--delete-older-than` 回收，但是**必须由对应用户运行**。

常见 GC Roots 来源

```
ls /nix/var/nix/gcroots/auto
```

 Bash

3. 其他使用 profile 的工具：

- `~/.local/state/nix/profiles/home-manager-N-link`

例如独立版本的 home-manager 也是调用 `nix-env` 来管理 profile 的⁴。可由 `home-manager expire-generations` 回收。

⁴见 home-manager 源码中的 [home-manager/home-manager](#) 文件。

常见 GC Roots 来源

```
ls /nix/var/nix/gcroots/auto
```

 Bash

4. nix-build/nix build 产生的 GC roots:

- result, result-bin, result-{OUTPUT_NAME}
- 或由 --out-link 选项指定的名称

5. nix-direnv 产生的 GC roots:

- .direnv/flake-profile* 和 .direnv/flake-inputs/*-source
- .direnv/nix-profile*

这些 GC roots 通常不会被回收，除非手动删除这些连接。

也有 `nix-sweep` 和 `nh clean` 这样的工具能够清理这些 GC roots。

现有工具

- `nix-collect-garbage` 只能回收 profile 相关的 GC roots，且选项很少。
- `nix-sweep` 和 `nh clean` 可以清理所有 GC roots，且选项较多，问题解决？

现有工具

- `nix-collect-garbage` 只能回收 profile 相关的 GC roots，且选项很少。
- `nix-sweep` 和 `nh clean` 可以清理所有 GC roots，且选项较多，问题解决？

其实现在基本解决了，只是在 `angrr` 被设计出来的时候还没有。

Flake 的问题

- 想象你有一个 flake，它定义好了开发环境，且 lock 住了所有依赖。
- 虽然你每天都在这个 flake 下工作，但环境已经很久很久没有变更了。
- 如果根据 GC roots 的**修改时间**来清理它们，这种一直被使用，但很久没变更的开发环境就会被清理掉 🥲。

angrr 的解决方案

提供一个特别的 direnv 库⁵，项目中无需任何配置，只要载入 .envrc，就会自动 touch 项目中**所有的** GC roots。

也提供了选项精细化控制被更新的 GC roots。

⁵<https://github.com/linyinfeng/angrr/blob/main/direnv/angrr.sh>

示例

项目 README⁶ 和 NixOS options 的文档中提供了配置示例。

```
1 {  
2   services.angrr = {  
3     enable = true;  
4     settings = { ... };  
5   };  
6   nix.gc.automatic = true;  
7   programs.direnv.enable = true;  
8 }
```



⁶<https://github.com/linyinfeng/angrr/blob/main/README.md>

示例 - 临时 GC roots

在 settings 中定义你想要的清理策略。

```
1 settings = {  
2     temporary-root-policies = {  
3         direnv = {  
4             path-regex = "/\\\\.direnv/";  
5             period = "2weeks";  
6         };  
7     };  
8 };
```



清理已经有两周没有使用的 .direnv/ 目录中的 GC roots。

示例 - 临时 GC roots

```
1 settings = {  
2     temporary-root-policies = {  
3         result = {  
4             path-regex = "/result[^/]*$";  
5             period = "3days";  
6         };  
7     };  
8 };
```



清理已经有三天没有使用的 result*。

示例 - Profile

```
1 settings.profile-policies = {  
2     system = {  
3         profile-paths = [ "/nix/var/nix/profiles/system" ];  
4         keep-since = "2weeks";  
5         keep-latest-n = 5;  
6         keep-current-system = true;  
7         keep-booted-system = true;  
8     };  
9 };
```



清理系统 profile，保留当前的（强制），最近两周内的，最新的五个，当前启动的，和当前正在运行的系统。

示例 - Profile

```
1 settings.profile-policies = {  
2     user = {  
3         profile-paths = [  
4             "~/.local/state/nix/profiles/profile"  
5             "/nix/var/nix/profiles/per-user/root/profile"  
6         ];  
7         keep-since = "1d";  
8         keep-latest-n = 1;  
9     };  
10 };
```



清理用户 profile，保留当前的（强制），最近一天内的，和最新的一个。

示例

```
1 settings.touch = {  
2     project-globs = [  
3         "!.git"  
4         "!target" "!node_modules"  
5     ];  
6 };
```



你还可以配置载入 `.envrc` 时要 touch 的 GC roots。例如忽略 `.git`（默认），`node_modules` 等目录以加快这个过程。或用它忽略你觉得不值得保留的 GC roots。

示例

配置好了，如何运行？

加入 NixOS 配置并 switch 后，无须做任何事。nix-gc.service 运行前会自动运行 angrrr.service。这个过程是无感的。

- nix.gc.dates 默认为 ["03:15"]，即每天凌晨 3:15 运行。

当然，你也可以手动运行 `angrr run` 执行清理策略⁷。

⁷默认情况下，作为 root 用户运行时，`angrr run` 执行整个系统范围的清理；否则只清理当前用户的 GC roots。

更多信息

CLI 文档和手册页：

```
1 man 1 angr # CLI interface
2 man 5 angr # configuration file
3 angr --help
```



NixOS options:

- `man configuration.nix` 并按 / 搜索 `angr`。
- [NixOS Search - Options - angr](#)

杂谈 - angr 与 nix-direnv

nix-direnv 在 [PR #631](#) 中加入了自动 touch .direnv 中的 GC roots 的功能。

- 因此 nix-sweep 和 nh clean 对 .direnv 也能达到类似 angr 的效果了。
- 不同的是，nix-direnv 只 touch 它创建的 GC roots，而 angr touch 项目目录中的所有 GC roots，且可配置。

杂谈 - angr 与 nix-direnv

nix-direnv 在 [PR #631](#) 中加入了自动 touch .direnv 中的 GC roots 的功能。

- 这个 PR 是 10 月 27 日提交的，直到做这个幻灯片时我才知道 🤔。
- angr 加入这个功能是在 9 月 25 日的 0.1.2 版本。Nixpkgs 里的更新 PR 一直挂着没有人 review，直到 NixOS 25.11 快要分叉了我才想起这事来，结果还是没有进 25.11。
- 结果 nix-direnv 的更新弥补了这一点 😊。

杂谈 - angr 与 Nixpkgs

关于放到 nixpkgs 中。

- 实际上我本来并无放到 nixpkgs 的计划。
- [SuperSandro2000](#) 不知为啥看到了这个项目，提 issue⁸ 建议我把它放到 Nixpkgs 里。然后我拖延症犯了，过了好几个月才提交 PR。

⁸<https://github.com/linyinfeng/angr/issues/3>

杂谈 - angrrr 与 Nixpkgs

关于放到 nixpkgs 中。

- 由于名字取了个“a”开头的，在 NixOS 25.11 release notes 新模块里排第二个，突然就更多人知道了，多了一些用户。

New Modules

- Added nixos-init, a Rust-based bashless initialization system for systemd initrd. This allows to build NixOS systems without any interpreter. Enable via `system.nixos-init.enable = true;`.
- [angrr](#), a service that automatically cleans up old auto GC roots. Available as [services.angrr](#).

如果想你的软件火起来，就取个“a”开头的名字贡献加到 nixpkgs 吧 😊 （不是

杂谈 - angr 与 Nixpkgs

0.2.0 版本前，angr 只专注于管理 `result/.direnv` 相关的临时 GC roots。

- 用户多了后，有用户希望它也能管理 profile⁹，因此我又加了一堆功能，演变成了现在的样子，既可以有 `temporary-root-policies` 又可以有 `profile-policies`，且高度可配置。

⁹<https://github.com/linyinfeng/angr/issues/30>

杂谈 - 软件开发风格

我比较倾向于做通用可配置的工具，然后可能提供一个不错的示例配置，不对用户的使用场景做过多假设。

这也是 `angrr` 有些不同于 `nix-sweep` 和 `nh clean` 的地方，也是为什么有了这两个工具，我仍然没有删库跑路，并觉得可以讲讲 `angrr` 的原因。

杂谈 - 软件开发风格

类似的还有 NixOS CN 群里的 commit-notifier¹⁰，虽然它的主要任务是发送与 Nixpkgs 相关的通知，但它其实非常通用，没有任何与 Nixpkgs 耦合的地方。可以通过配置用它发送任何 GitHub 仓库的通知¹¹，比如大家也很关心的 [nixos/nix](https://github.com/nixos/nix)。

可能实际上并没有人注意到过这一点 🤔

¹⁰<https://github.com/linyinfeng/commit-notifier>

¹¹可以联系 bot 的管理员（我）添加仓库，或者部署自己的 bot。

杂谈 - 国际化

之前 angr 的 direnv 脚本中有这么一句：

```
1 runtime_formatted=$(printf "%.3f" "$runtime")
```



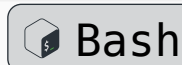
"\$runtime" 将被 bash 展开为形如 0.027721948 的字符串。

你能看出问题所在吗？

杂谈 - 国际化

之前 angr 的 direnv 脚本中有这么一句：

```
1 runtime_formatted=$(printf "%.3f" "$runtime")
```

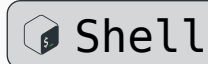


"\$runtime" 将被 bash 展开为形如 0.027721948 的字符串。

你能看出问题所在吗？

在德语系统上这段代码就会出问题 ([linyinfeng/angrr#36](#)):

```
1 $ LC_ALL=de_DE.UTF-8 printf "%.3f" 0.027721948
2 bash: printf: 0.27721948: Ungültige Zahl.
3 0,000
```



杂谈 - Nix GC 的一些问题

最后，在开发 angr 的过程中，我也意识到 Nix 在 GC 上存在的一些问题：

1. 并非所有的 GC roots 在 GC 时都总是可见

为什么这对单机来说是一个问题？难道 GC roots 能长腿跑了么？

杂谈 - Nix GC 的一些问题

最后，在开发 `angrr` 的过程中，我也意识到 Nix 在 GC 上存在的一些问题：

1. 并非所有的 GC roots 在 GC 时都总是可见

为什么这对单机来说是一个问题？难道 GC roots 能长腿跑了么？

- 比如，有些用户可能想要加密自己的 home 目录；甚至 `systemd-homed` 默认就加密用户的 home 目录。
- 一旦用户登出，home 目录被卸载，用户的 GC roots 就无法被 Nix 访问到了。
- 这种情况下执行 `nixos-collect-garbage` 会回收用户想要保留的 store path 😞。

杂谈 - Nix GC 的一些问题

最后，在开发 angr 的过程中，我也意识到 Nix 在 GC 上存在的一些问题：

2. 混乱的 GC roots 组织（小问题）

我们已经提过，Nix 会在 `/nix/var/nix/{gcroots,profiles}` 下查找 GC roots。但实际上 `profiles` 本来就有有一个软链接在 `gcroots` 目录下：

```
/nix/var/nix/gcroots/profiles -> /nix/var/nix/profiles
```

并且你会发现，所有的 profile generation 的 GC roots 还会同时出现在 `/nix/var/nix/gcroots/auto` 目录下。

一个 profile generation 会在各种地方被 Nix GC 看到三次，何意味。

👋 谢谢!



欢迎试用，提交 issue 与 PR。